

Object-oriented Programming in OCaml

Programming Language & Compiler

School of Computer Science, Wuhan University

目录

1	References	1
2	Preliminaries	1
2.1	Introduction	1
2.2	Types in OCaml	1
2.2.1	Variants	1
2.2.2	Recursive Variants	2
2.2.3	Parameterized Variants	2
2.2.4	Polymorphic Variants	3
2.2.5	Record	4
2.3	Mutability	5
2.3.1	Mutable	5
2.3.2	Refs	6
2.4	Error Handling	6
2.4.1	Exception	6
2.4.2	Semantic of Exception	8
3	Module System	9
3.1	Introduction	9
3.2	Modules	9
3.2.1	Module Definition	9
3.2.2	Operational Semantics of Module	11
3.2.3	Module Type Definition	12
3.2.4	Semantics of Signature	14
3.2.5	Scope	15
3.3	Data Structure in OCaml	16
3.3.1	Stack	16
3.4	Include	17
3.4.1	Include Definition	17
3.4.2	Includes vs. Open	18
3.5	Functors	18
3.5.1	Functor Definition	18
3.5.2	Functor Type Definition	20
3.5.3	Map	20

3.5.4	Set	21
3.5.5	“Functor”	21
4	Class and Object	22
4.1	Introduction	22
4.2	Class Definition	22
4.2.1	Basic Class and Object	22
4.2.2	Immediate Object	24
4.2.3	Polymorphic Class	25
4.3	Inheritance	26
4.3.1	Virtual Method	26
4.3.2	Inherit	27
4.3.3	Class Interface	29
4.3.4	Privacy	29
4.3.5	Oo Module	30
5	Extend	30
5.1	Compile	30
5.1.1	Dune	30
5.2	Test	30
5.2.1	OUnit2	30

1 References

本次实验涉及到的知识点很多，讲义篇幅较长，并且大量参考了下面这些资料（有些时候讲义仅仅是这些资料的汉化版）。你可以参考这些资料进行更详细更系统的学习：

- Cornell cs3110 Videos [Bilibili](#) 康奈尔大学函数式编程课程，200 多集每集仅 3 到 10 分钟；课程涵盖知识面很广，从函数式编程到编译技术；还可以锻炼下英语听力
- Cornell cs3110 Textbook [Website](#) 课程配套讲义，包含部分视频内没有讲到的内容；附有课后练习
- OCaml 官方文档 [Website](#) 官方的语言特性文档以及 API 文档
- OCaml 用户手册与官方文档 [Website](#) Inria 编写的用户手册，在面向对象编程这一部分讲授的更加清晰
- Stanford cs242 notes & assignments [Website](#) 斯坦福的程序语言理论和编译技术课，没有公开讲课录像，仅有部分讲义以及课后作业
- Types and Programming Languages [Book](#) 《类型与程序设计语言》

重要

如果你认为讲义的某些部分出错了，或者有改进建议（比如废话太多或者语句不通顺），可以反馈给助教，这可以帮助改进这门课程（也可能对下一届的学弟学妹很有用）🙏

2 Preliminaries

2.1 Introduction

这一部分是前置知识，你在后面的面向对象编程会大量运用这一部分的内容。

2.2 Types in OCaml

注意

这一节的内容实际上是 TaPL Chapter 11 Simple Extensions 中的 11.10 Variants 的 OCaml 实现，你可以先复习一下这个章节。Variants vs. Datatypes 这个小节特别介绍了 TaPL 和 ML 语言家族中变体的不同之处。

2.2.1 Variants

OCaml 提供关键字 `type` 来定义变体（Variants）：

```
type t = C1 [of t1] | ... | Cn [of tn]
```

`t` 是变体名称，`C1...Cn` 是类型构造器（constructor），`t1...tn` 是构造器对应的类型。当类型构造器是 `Unit` 类型时，变体就近似于其他高级语言中的枚举类型。你可以尝试定义一个日期类型 `days`，记录一周内的七天：

⚠ 小心

OCaml 中要求变体以小写字母开头。同样地，`Unit` 类型在 OCaml 中是 `unit`，而非 `Unit`，后者是 OCaml 中的一个模块。为了和 TaPL 相对应，讲义正文中的类型都以大写字母开头。

```
(* days 类型定义了一周的前三天 *)
type days = Mon of unit | Tue of unit | Wed of unit;;
```

当类型构造器都是 Unit 类型的时候，OCaml 允许你忽略所有的 `of t1, ..., of tn`，写成下面这种形式也是可以的（此时和其他高级语言的枚举类型非常相似）：

```
type days = Mon | Tue | Wed;;
let x = Mon;;
```

```
> val x : days = Mon
```

类型构造器也可以是其他类型，各个类型构造器的类型可以不同：

```
(* 这里的两个类型分别表示直角坐标系和极坐标系下的点 *)
(* int * int 代表元组 *)
type shape = Rect of int * int | Circle of int;;
let x = Rect(2, 3);;
let y = Circle(5);;
```

```
> val x : shape = Rect (2, 3)
> val y : shape = Circle 5
```

当类型构造器是一个已经存在的类型时，变体也可以作为类型别名使用：

```
(* 在当前上下文中给 int 一个类型别名 *)
type intalias = int;;
let x = 9;;
```

```
val x : intalias = 9
```

2.2.2 Recursive Variants

你可以递归地定义变体类型，通过在类型构造器中加入当前正在定义的变体名称实现递归类型，以实现一个链表类型为例：

```
type linked = Empty | Cons of int * linked;;
let lst = Cons(1, Cons(2, Cons(3, Empty)));;
```

```
> type linked = Empty | Cons of int * linked
> val lst : linked = Cons (1, Cons (2, Cons (3, Empty)))
```

2.2.3 Parameterized Variants

当你定义了一个对参数类型没有约束的函数：

```
let return_number_2 = fun x -> 2;;
```

```
> val return_number_2 : 'a -> int = <fun>
```

容易发现输出中参数 `x` 的类型是 `'a`，而非常见的 `int`、`string` 等 OCaml 内置的基础类型。`'a` 是 OCaml 中显式的类型变量，用来表示参数 `x` 的多态类型。变体允许你使用类型变量给类型构造器添加参数。上面实现的链表类型仅仅支持整数 `Int` 作为链表内的元素，你可以使用类型变量改进链表类型：

```
type 'a linked = Empty | Cons of 'a * 'a linked;;
```

此时，`linked` 不再是一个类型，而是一个类型构造器：接收一个类型作为参数，然后返回这个类型链接组成的链表类型：

```
let lst = Cons(1, Cons(2, Cons(3, Empty)));;
let lst_str = Cons("0", Cons("Caml", Empty));;
```

```
> val lst : int linked = Cons (1, Cons (2, Cons (3, Empty)))
> val lst_str : string linked = Cons ("0", Cons ("Caml", Empty))
```

此时的 `lst` 具有类型值 `int linked`，`lst_str` 具有类型值 `string linked`。

上面的案例中类型构造器 `linked` 的参数 `'a` 是隐式确定的，类型系统帮助你完成了参数类型的推理工作。你也可以显式地指定具体类型，就像把 `int` 作为参数传递给 `linked` 那样：

```
(* 通过类型注解指定类型 *)
let lst : int linked = Cons(1, Cons(2, Cons(3, Empty)));;
(* 使用类型别名 *)
type int_linked = int linked;;
type str_linked = string linked;;
```

2.2.4 Polymorphic Variants

多态变体（Polymorphic Variants）是 OCaml 中另一个强大的特性。通过使用多态变体，你在实现一个具有多态返回类型的函数时，无需提前定义好一个变体供函数使用：

```
let f = function
| 0 -> `Infinity
| 1 -> `Finite 1
| n -> `Finite (-n)
```

① 注意

上面的代码没有提前定义一个变体 `type number = Infinity | Finite of int`，而是在类型构造器前加上反引号 ` 并直接使用这个类型构造器。因此，在使用这个特点时，多态变体也可以被称为“匿名变体”——我们没有显式定义整个变体的名称，而是直接使用了变体包含的类型构造器。

输出是：

```
> val f : int -> [> `Finite of int | `Infinity ] = <fun>
```

`int` 表示函数 `f` 的参数类型，`[ ]` 内的部分是函数的返回类型。`>` 代表这是一个开放类型，除了 `[ ]` 内已经包含的构造器，还允许其他的构造器。

函数也可以使用多态变体作为参数类型，你可以设计一个函数 `f` 的反函数 `g`：

```
let g = function
| `Infinity -> 0
| `Finite 1 -> 1
| `Finite n -> -n;;
```

输出是：

```
val g : [< `Finite of int | `Infinity ] -> int = <fun>
```

< 代表**封闭类型**，只允许包含 [] 内的构造器。

开放类型相比于封闭类型更容易让人费解，为什么允许其他的构造器出现？多态变体的核心思想是**灵活性和可组合性**。开放类型允许你编写能够处理未完全指定构造器集合的代码，保留多态性，确保函数可以与其他代码组合使用：

```
let combined = `A 32 (* 类型为 [> `A of int] *)
let extended = `B "OCaml" (* 类型为 [> `B of string] *)
let lst = [combined; extended] (* 类型为 [> `A of int | `B of string] list *)
```

```
> val lst : [> `A of int | `B of string ] list = [ `A 32; `B "OCaml"]
```

2.2.5 Record

记录（record）是一组字段的集合。想要在 OCaml 中使用记录，你首先需要使用 `type` 关键字定义一个记录类型：

```
type record_name = {
  name: type;
  name: type;
}
```

`record_name` 是这种记录类型的名称，`name` 和 `type` 分别指字段的名称和类型。以定义一个宝可梦记录类型为例：

```
type pokemonType = {
  (* 宝可梦的名字 *)
  name: string;
  (* 宝可梦的血量种族值 *)
  hp: int;
  (* 宝可梦的属性 *)
  ptype: string;
}
```

然后你可以定义一个宝可梦类型的记录，参考：<https://wiki.52poke.com/wiki/%E5%8F%AF%E8%BE%BE%E9%B8%AD>



```
let pokemon = { name = "psyduck"; hp = 50; ptype = "water" };;
```

```
val pokemon : pokemonType = {name = "psyduck"; hp = 50; ptype = "water"}
```

💡 提示

还有一种数据结构叫做元组 (Tuple)。记录和元组都出现在 TaPL Chapter 11 中，你可以参考书本和 OCaml 文档（或者其他资料）详细了解它们。

2.3 Mutability

2.3.1 Mutable

命令式编程和函数式编程各有优劣，OCaml 允许有效地将两者结合起来。因此 OCaml 也支持你使用一些“可变”的东西，方便你更好的开发某个项目。

OCaml 提供关键字 `mutable` 标记记录中的某个字段，这会导致该字段可以变化，语法：

```
type record_name = {  
  mutable name: type  
}
```

以宝可梦类型为例，为其添加一个可变的 `level` 字段：

```
type pokemonType = {  
  name: string;  
  hp: int;  
  ptype: string;  
  mutable level: int;  
};;
```

再定义一个 1 级的可达鸭：

```
let pokemon = { name = "psyduck"; hp = 50; ptype = "water"; level = 1 };
```

```
> val pokemon : pokemonType =  
  {name = "psyduck"; hp = 50; ptype = "water"; level = 1}
```

使用 `<-` 运算符改变可达鸭的等级为 2 级：

```
pokemon.level <- 2;;  
pokemon.level
```

```
> - : unit = ()  
> - : int = 2
```

观察到，可达鸭的等级增长到了 2。



2.3.2 Refs

当使用 `let ... = ...` 将值绑定到名称时, 此名称-值绑定是不可变的, 因此无法修改。OCaml 提供引用 (reference) 来帮助处理可变数据。关键字 `ref` 用来定义引用; `!` 用来解引用, 取出引用里面的值; `:=` 用来更新引用内包含的值。

```
(* 定义一个引用 *)
let r = ref ...
(* 解引用 *)
let ... = !r
(* 修改引用值 *)
r := ...
```

💡 提示

前端框架 Vue 里的“响应式变量”关键字也是 Ref, 你可以把两者做个对比。

用一个案例辅助你了解引用:

```
let r = ref 5;;
let x = !r;;
r := 10;;
let y = !r;;
```

```
> val r : int ref = {contents = 5}
> val x : int = 5
> - : unit = ()
> val y : int = 10
```

更新是作为副作用完成的, 返回 `unit` 类型。

通过输出你应该可以发现, 在 OCaml 中, 引用实际上是具有单个可变字段的记录:

```
type 'a ref = { mutable contents : 'a; }
```

`'a ref` 是一个记录类型, 它具有一个名为 `contents` 的单个字段, 该字段用 `mutable` 关键字标记。

2.4 Error Handling

2.4.1 Exception

OCaml 提供异常 (Exception) 作为处理程序执行过程中错误或特殊情况的机制之一。在 OCaml 中使用异常可以被划分为三个步骤: 1. 定义异常构造器; 2. 使用异常构造器构造一个异常值; 3. 在某些情况下抛出异常。定义无参数异常构造器的语法:

```
exception E
```

`E` 是异常构造器名称。异常构造器也可以携带参数:

```
exception E of t
```

`E` 是异常构造器名称, `of t` 指明异常携带了参数及参数对应的类型, `t` 是类型构造器。你可以尝试自己定义下面几个异常构造器:

```
(* 无参数的异常 *)
exception MyError
(* 返回错误码的异常 *)
exception CodeError of int
(* 返回错误信息的异常 *)
exception MsgError of string
```

然后你可以通过使用构造器得到一个异常值, 每个异常值都是 `exn` 类型:

```
let myerror = MyError;;
let codeerror = CodeError 404;;
let msgerror = MsgError "Bad Gateway";;
```

```
> val myerror : exn = MyError
> val codeerror : exn = CodeError 404
> val msgerror : exn = MsgError "Bad Gateway"
```

OCaml 标准库预定义多种异常构造器: 普通异常 `Failure`、除零异常 `Division_by_zero`、参数非法异常 `Invalid_argument` 等。在开发时你也可以直接使用它们。

抛出异常使用 `raise` 关键字: `raise e`。 `e` 是异常值。

OCaml 提供 `try ... with` 结构用于异常处理, 类似于其他语言中的 `try-catch` :

```
try
  (* 可能抛出异常的表达式 *)
with
| Exception1 -> (* 处理异常1 *)
| Exception2 -> (* 处理异常2 *)
| _ -> (* 处理所有其他异常 *)
```

综合上面的知识, 你可以在计算阶乘时检查参数的正负性, 提升代码的健壮性:

```
exception NegativeNumber

let factorial n =
  if n < 0 then raise NegativeNumber
  else if n = 0 then 1
  else n * factorial (n - 1)

let () =
  try
    let result = factorial (-5) in
    Printf.printf "Result: %d\n" result
  with
  | NegativeNumber ->
    Printf.eprintf "Error: Negative input to factorial!\n"
  | exn ->
    Printf.eprintf "Unexpected error: %s\n" (Printexc.to_string exn);
    raise exn (* 重新抛出未处理的异常 *)
```

2.4.2 Semantic of Exception

① 注意

这一部分与 TaPL Chapter 14 Exceptions 相关，你可以先复习这个章节

→ exceptions		Extends $\lambda_{\rightarrow}$ (9-1)	
New syntactic forms $t ::= \dots$ $\text{raise } t$ $\text{try } t \text{ with } t$		terms: <i>raise exception</i> <i>handle exceptions</i>	
New evaluation rules $(\text{raise } v_{11}) t_2 \rightarrow \text{raise } v_{11}$ (E-APPRAISE1) $v_1 (\text{raise } v_{21}) \rightarrow \text{raise } v_{21}$ (E-APPRAISE2) $\frac{t_1 \rightarrow t'_1}{\text{raise } t_1 \rightarrow \text{raise } t'_1}$ (E-RAISE) $\text{raise } (\text{raise } v_{11}) \rightarrow \text{raise } v_{11}$ (E-RAISERAISE)		$\text{try } v_1 \text{ with } t_2 \rightarrow v_1$ (E-TRYV) $\text{try raise } v_{11} \text{ with } t_2 \rightarrow t_2 v_{11}$ (E-TRYRAISE) $\frac{t_1 \rightarrow t'_1}{\text{try } t_1 \text{ with } t_2 \rightarrow \text{try } t'_1 \text{ with } t_2}$ (E-TRY)	
New typing rules $\frac{\Gamma \vdash t_1 : T_{\text{exn}}}{\Gamma \vdash \text{raise } t_1 : T}$ (T-EXN) $\frac{\Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : T_{\text{exn}} \rightarrow T}{\Gamma \vdash \text{try } t_1 \text{ with } t_2 : T}$ (T-TRY)		$\boxed{t \rightarrow t'}$ $\boxed{\Gamma \vdash t : T}$	

Figure 14-3: Exceptions carrying values

这里的 $\text{raise } t$ 是一个项构造器， t 是异常携带的异常信息， t 和 OCaml 中的异常值相对应。E-APPRAISE 阐释了当包含异常时项的求值顺序：当遇到项抛出一个异常，程序应当立刻把该异常作为返回值，这与 OCaml 中的异常求值顺序相符，当我们写出下面的代码：

```
exception A
exception B
let _ = raise A in raise B;;
```

```
> Exception: A.
```

程序立即抛出了异常 A，因为 let 表达式中绑定表达式必须在主体表达式之前求值，in 前面的部分被更早地求值。同样地，根据按值调用（call-by-value），参数会在函数体之前被求值，下面的代码也会抛出异常 A：

```
(raise B) (raise A)
```

```
> Exception: A.
```

E-TRY、E-TRYV 和 E-TRYRAISE 定义了异常处理器（在 OCaml 中同样是 $\text{try } \dots \text{ with } e$ ）的实际行为：不断归约异常处理器的主体，一旦一个异常处理器的主体被规约为一个值，那么可以丢弃这个异常处理器返回这个值，否则抛出携带信息的异常。

类型 T_{exn} 对应于 OCaml 中的类型 `exn`。之前我们发现，得到的所有异常值都是 `exn` 类型，这是因为 OCaml 把 T_{exn} 用变体进行实现，使得异常可以携带更多类型的信息。T-EXN 说明异常信息 t 有类型，但是 raise 可以是上下文推理出的任何类型，这有助于减少对其他表达式类型规则的修改，但是同样给类型检查带来了难度。

3 Module System

3.1 Introduction

模块化编程 (Modular Programming) 是一种将大型、复杂的编程任务分解为多个独立的、更小、更易于管理的子任务或模块的过程。这些模块可以像构建块一样拼凑在一起, 创建更大的应用程序。OCaml 提供 **模块系统 Module System** 以支持模块化编程。模块化编程通常具备如下特征:

- **命名域 Namespace** 命名域限制名称作用的范围, 来自两个命名域的相同名称可以具备不同的语义。
- **抽象 Abstraction** 抽象隐藏模块内部的实现细节, 并提供了模块的使用接口, 避免使用者滥用模块内部的信息, 方便其他程序员在模块的基础上进行开发。
- **代码复用 Code Reuse** 模块可以在多个 OCaml 源代码中被引用, 从而无需把业务代码进行重复复制。

⚠ 小心

虽然模块化编程和面向对象编程 (Object-oriented Programming, OOP) 在某些特点上十分相似, 但是二者并不等同。一个足够“显眼”的区别是: 一份 OCaml 源文件只有“一个”独特的模块, 你使用 `<模块名>.<方法名>` 的方法调用它, 但是却可能有多个类的对象实例 (Object), 你可以在每个实例上调用类的方法, 比如 `<对象名>.<方法名>`。OCaml 中的模块系统非常强大, 以致于你可能很少使用 Class 而大量使用 Module。

3.2 Modules

3.2.1 Module Definition

OCaml 规定使用关键字 `module` 定义模块, 语法如下:

```
module ModuleName = struct
  module_items
end
```

`ModuleName` 是模块名。 `module_items` 是模块内部的项, 包括 `let` `type` 等多种定义语句或者嵌套定义语句。

⚠ 小心

模块名必须以大写开头。最好使用**大驼峰命名法**命名模块。

以定义一个 `Color` 模块为例:

```
module Color = struct
  (* 定义颜色类型 *)
```

```

type t = Red | Green | Blue

(*初始颜色*)
let red = Red
let green = Green
let blue = Blue

(* 将颜色转换为字符串 *)
let to_string = function
  | Red -> "Red"
  | Green -> "Green"
  | Blue -> "Blue"

(* 按顺序获取下一个颜色 *)
let next = function
  | Red -> Green
  | Green -> Blue
  | Blue -> Red
end

let red = Color.red

let () =
  print_endline (Color.to_string red);
  print_endline (Color.to_string (Color.next red))

```

在 utop 里输入 `#use "Color.ml"` 执行这段代码，输出：

```

> module Color :
sig
  type t = Red | Green | Blue
  val red : t
  val green : t
  val blue : t
  val to_string : t -> string
  val next : t -> t
end
> val red : Color.t = Color.Red
> Red
> Green

```

💡 提示

当你使用编辑器编写了一个 OCaml 源程序并保存为 .ml 文件后，如果不想手动把文件里的代码段粘贴到 utop 中再执行，可以在文件所在的目录下打开 utop，输入 `#use filename`，这个命令的效果和手动在 utop 里输入或者复制粘贴代码是一样的。我们刚刚编写了一个 `Color.ml`，就输入 `#use "Color.ml";;`。

另一个更准确的模块定义的语法描述是：

```
module ModuleName = module_expression
```

`struct` 表达式仅仅是 `module_expression` 的一种。我们可以给予某个 OCaml 模块一个别名，比如：`module MyList = List`。此时，你就可以使用 `MyList.<item>` 来调用 `List` 上的项，比如：

```
module MyList = List
let arr = [1; 2; 3;]
let res = MyList.fold_left ( + ) 0 arr;;
```

```
> module MyList = List
> val arr : int list = [1; 2; 3]
> val res : int = 6
```

模块的定义允许为空的 `struct` 表达式:

```
module Mod = struct end
```

OCaml 同样支持嵌套模块 (Nested Module)，你可以在一个模块里面定义并使用另一个模块:

```
module M = struct
  module Inner = struct
    let x = 42
  end
  let y = Inner.x
end
```

3.2.2 Operational Semantics of Module

回忆之前的内容：我们使用 `let` 把某个值绑定到变量名上。当你在 `utop` 输入:

```
let x = 1 + 1;;
```

输出:

```
> val x: int = 2
```

容易观察到，表达式 `1 + 1` 会被求值后绑定给 `x`，而不会把还未计算的完整表达式 `1 + 1` 绑定给 `x`。这意味着 `let <term> = <expr1> in <expr2>` 中的 `<expr1>` 会在代换前进行求值，这与 TaPL 中 `let` 的求值规则相符:

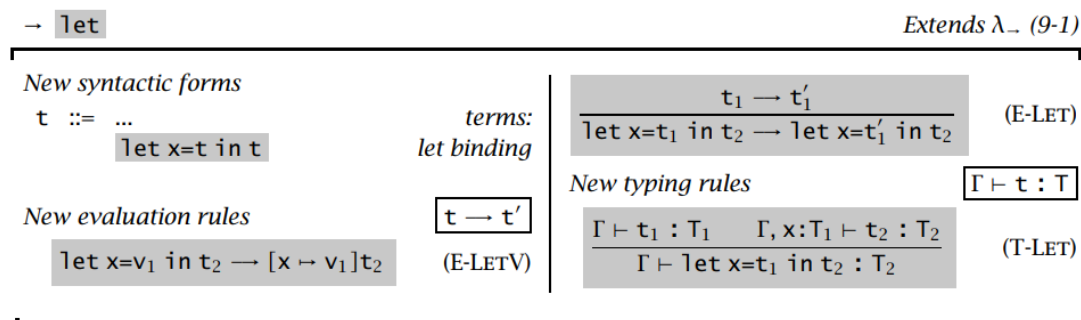


Figure 11-4: Let binding

💡 提示

如果你忘记了这部分知识，回顾 TaPL 中的概念：操作语义和求值顺序 (Chapter 3 Untyped Arithmetic Expressions, Chapter 5 The Untyped Lambda Calculus)、序列 (Chapter 11 Simple Extensions)、let 绑定 (Chapter 11 Simple Extensions)

模块的动态语义与上面类似，可以把 `Module` 和 `let` 相对应，`let` 把一个值绑定到一个变量名上，`Module` 把一个**模块值** 绑定到变量名上，这意味着 `module_expression` 会被首先求值再做绑定，因此这样的定义是合法的：

```
module M = struct
  let x = 0
  let y = x
end
```

但是这样的定义是非法的，因为 `x` 在被求值时并没有被绑定：

```
module M = struct
  let y = x
  let y = 0
end
```

```
> Error: Unbound value x
```

⚠ 小心

把 `module` 和 `let` 做类比只是为了帮助你更好的理解模块的操作语义。模块值和其他的值并不相同，不可以用模块值等价地当做值使用，模块值：

1. 不可以使用 `let` 进行绑定。
2. 不可以作为函数参数或者返回值。

3.2.3 Module Type Definition

OCaml 使用签名 (signature) 来标识一个模块的类型。在上面的 `Color` 模块案例中，我们可以发现 OCaml 做出了这样的类型推断并打印到输出：

```
> module Color :
sig
  type t = Red | Green | Blue
  val red : t
  val green : t
  val blue : t
  val to_string : t -> string
  val next : t -> t
end
```

当我们手动指定模块的签名时，签名还可以作为模块的接口 (interface)，就像 C++ 中的头文件那样，指定模块的哪些组件可以由外部访问，以及暴露哪些类型。模块签名的定义语法如下：

```
module type ModuleType = sig
  specifications
end
```

或者

```
module type ModuleType = module_type_expression
```

`specifications` 是模块中项的类型注释, 包括 `val` 声明、类型和异常的定义以及嵌套类型定义。现在, 我们可以编写一个模块类型 `ColorType`, 并使用运算符 `:` 指定模块 `Color` 的类型是 `ColorType` :

```
module type ColorType = sig
  (* 定义颜色类型 *)
  type t

  (* 初始颜色 *)
  val red : t

  (* 将颜色转换为字符串 *)
  val to_string : t -> string

  (* 按顺序获取下一个颜色 *)
  val next : t -> t
end

module Color : ColorType = struct
  type t = Red | Green | Blue

  let red = Red
  let green = Green
  let blue = Blue

  let to_string = function
    | Red -> "Red"
    | Green -> "Green"
    | Blue -> "Blue"

  let next = function
    | Red -> Green
    | Green -> Blue
    | Blue -> Red
end
```

① 注意

注释的位置从模块里面转移到了签名里面。这些注释是签名中名称规范的合理组成部分。它们描述了项的抽象行为。

为了处理嵌套模块, 签名也具备嵌套结构, 在下面的示例中, `T` 指定必须存在一个名为 `Inner` 的内部模块, 其模块类型为绑定后的签名 `X`

```
module type X = sig
  val x : int
end

module type T = sig
  module Inner : X
  val y : int
end

module M : T = struct
  module Inner : X = struct
    let x = 42
  end
end
```

```

end
let y = Inner.x
end

```

签名并不需要包含模块中所有的项, 一些模块内部的函数可以不在签名中定义, 这些函数就不会暴露给外部:

```

module type ModType = sig
  val f : int -> int
  val g : int -> int
end
module Mod : ModType = struct
  let f x = x + 1
  let g x = x * 2
  let h x = x - 1
end
let result = Mod.f 3 + Mod.g 4
(* let result = Mod.h 5 will fail*)

```

在这个例子中, 调用模块中的 `f` 和 `g` 是成功的, 但是如果想要调用 `h` 就无法通过静态检查:

```
> Error: Unbound value Mod.h
```

3.2.4 Semantics of Signature

签名作为模块类型, 就是 OCaml 类型系统中的一员。不同于 Python 和 Javascript, OCaml 的类型系统是静态的 (Static), 在编译时做严格的类型检查, 在编译后进行类型擦除。因此签名只在编译时发挥作用:

1. 检查模块中项的实现是否与签名中的类型定义相符
2. 没有出现在签名的项不会向外部暴露

(Sub Refl)	(Sub Trans)	(Val Subsumption)
$\frac{\Gamma \vdash A}{\Gamma \vdash A <: A}$	$\frac{\Gamma \vdash A <: B \quad \Gamma \vdash B <: C}{\Gamma \vdash A <: C}$	$\frac{\Gamma \vdash a : A \quad \Gamma \vdash A <: B}{\Gamma \vdash a : B}$
(Type Top)	(Sub Top)	(Sub Arrow)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash Top}$	$\frac{\Gamma \vdash A}{\Gamma \vdash A <: Top}$	$\frac{\Gamma \vdash A' <: A \quad \Gamma \vdash B <: B'}{\Gamma \vdash A \rightarrow B <: A' \rightarrow B'}$

子类型 (Subtyping) 直观地刻画了类型之间的包含关系, 解决了 OOP 中的多态问题。如果你还记得 TaPL 中的子类型的知识, 你可以更精确地描述签名的静态语义:

1. 模块类型注解 $(M : T)$ 的有效性条件是: 当模块 `M` 的模块类型是 `T` 的子类型时成立。在后续的类型检查中, $(M : T)$ 的模块类型将被看作 `T`。
2. 模块类型 `S` 作为 `T` 的子类型需满足: `S` 中的定义集合是 `T` 定义集合的超集。

**小心**

子类型和高级程序设计中的“继承”不是一个概念

3.2.5 Scope

在一个模块内定义的所有项的作用域是这个模块内部，想要在外部使用项需要满足：

1. 项已经在签名中被声明
2. 使用 `<模块名>.<项>` 的格式使用

比如，对于下面这个案例：

```
module type SigM = sig
  val x : int
  val y : int
end;;
module M : SigM = struct
  let x = 5
  let y = 10
  let z = 15
end;;
```

如果我们直接在外部输入 `x`：

```
x;;
> Error: Unbound value x
```

如果输入 `M.x`：

```
M.x;;
> int = 5
```

输入 `M.z`：

```
M.z;;
> Error: Unbound value M.z
```

OCaml 提供了一个关键字 `open`，它可以把跟在 `open` 后面的模块在当前的作用域内“打开”。当调用打开后的模块中的项的时候，不需要再添加 `<模块名>.`，比如：

```
module type SigM = sig
  val x : int
  val y : int
end;;
module M : SigM = struct
  let x = 5
  let y = 10
  let z = 15
end;;

open M;; (* 打开模块 M *)

x;; (* 尝试使用x *)
```

此时，`x` 的绑定能够正确地被输出：

```
> int = 5
```

💡 提示

如果你接触过 C++，可以把 `open <模块>` 与 `using namespace <命名域>` 做个类比

`open` 在某些时刻可以简化你的编码量，如果你正在编写的某个模块大量的使用了某个别的模块，使用 `open` 可以减少你输入模块名的次数，就像：

```
module M = struct
  (* 打开了 List 和 String *)
  open List
  open String

  (* 不再需要 List.map 和 String.uppercase_ascii *)
  let uppercase_all = map uppercase_ascii
end
```

你还可以使用 `let ... in ...` 把 `open` 限定在一个更小的作用域中：

```
let lower_trim s =
  let open String in
  s |> trim |> lowercase_ascii (* open 只在 in 后面的区域中生效 *)
```

⚠️ 小心

即使这很方便，但是“无脑”打开所有的模块对软件开发是非常有害的，最好有节制的使用

3.3 Data Structure in OCaml

3.3.1 Stack

我们可以利用模块在 OCaml 中实现对栈进行封装。下面给出了使用 List 的一种实现：

```
module type Stack = sig
  type 'a t
  (* 当栈为空时抛出 Empty 异常 *)
  exception Empty
  (* 创建一个空栈 *)
  val empty : 'a t
  (* 判断栈是否为空 *)
  val is_empty : 'a t -> bool
  (* 将元素推入栈 *)
  val push : 'a -> 'a t -> 'a t
  (* 返回栈顶元素 *)
  val peek : 'a t -> 'a
  (* 弹出栈顶元素 *)
  val pop : 'a t -> 'a t
  (* 返回栈的大小 *)
  val size : 'a t -> int
```

```

(* 将栈转换为列表 *)
val to_list : 'a t -> 'a list
end

module ListStack : Stack = struct
  type 'a t = 'a list
  exception Empty
  let empty = []
  let is_empty = function [] -> true | _ -> false
  let push = List.cons
  let peek = function [] -> raise Empty | x :: _ -> x
  let pop = function [] -> raise Empty | _ :: s -> s
  let size = List.length
  let to_list = Fun.id
end

```

💡 提示

栈是一种后进先出的线性数据结构。你可以参考 <https://oi-wiki.org/ds/stack/>

3.4 Include

3.4.1 Include Definition

OCaml 提供了**模块包含**来解决面向对象编程中的继承问题，包含 (includes) 的语法如下：

```

(* 当模块 A 尝试包含模块 B 时，模块 A 的签名应当包含模块 B 的签名 *)
module type moduleASig = type
  include moduleBSig
  module_items
end

(* 模块 A 包含模块 B *)
module moduleA : moduleASig = struct
  include moduleB
  module_items
end

```

包含使得一个模块包含所有由另一个模块定义的项，以及一个模块签名包含所有另一个模块签名定义的项。其作用类似于把一个模块（签名）的内部代码在另一个模块（签名）的内部进行“复制粘贴”。比如：

```

(* 定义了一个“基类” People *)
module type PeopleSig = sig
  type person
  val create : string -> person
  val name : person -> string
end

module People : PeopleSig = struct
  type person = {name: string}
  let create n = {name = n}
  let name p = p.name
end

```

```
(* 定义了一个“派生类” TeachAssisant, TeachAssisant 包含 People 所有的项, 同时增加了一个
role 项*)
module type TeachAssisantSig = sig
  include PeopleSig
  val role : person -> string
end

module TeachAssisant : TeachAssisantSig = struct
  include People
  let role p = "Teaching Assistant"
end

let ella = TeachAssisant.create "Ella"
let () = Printf.printf "%s is a %s\n" (TeachAssisant.name ella)
(TeachAssisant.role ella)
```

```
> Ella is a Teaching Assistant
```

ella 创建成功, 并且成功地调用了 name 和 role 两个项。

3.4.2 Includes vs. Open

include 和 open 导致的效果看上去很相似, 但是二者截然不同: open 的作用是直接暴露目标模块的项, 允许省略模块名前缀, 但不改变当前模块的结构, 仅影响作用域; 而 include 则是将目标模块原样复制到当前模块中, 类似于代码展开, 从而成为当前模块的组成部分。这意味着 include 会实际改变模块的结构, 并允许通过后续定义覆盖被包含的内容, 而 open 仅影响编译时的名称解析。

3.5 Functors

3.5.1 Functor Definition

简单地说, OCaml 中的函子 (Functor) 是参数化的模块, 类似于以模块作为参数和返回值的一个“函数”。从一个简单的案例入手:

```
(* 定义一个非常简单的模块签名 *)
module type X = sig
  val x : int
end

(* 定义一个函子 IncX *)
module IncX (M: X) = struct
  let x = M.x + 1
end
```

```
> module type X = sig val x : int end
> module IncX : functor (M : X) -> sig val x : int end
```

OCaml 的类型系统推断 IncX 是一个函子, 接受模块签名为 X 的模块 M 作为参数, 返回的模块具有模块签名 sig val x : int end。你可以测试一下函子的功能, 定义一个模块 A 并且 A.x = 0, 如果函子正常工作, 那么使用函子创建出的模块 B 应该满足 B.x = A.x + 1 = 1 :

```
(* 首先定义一个模块 A *)
module A : X = struct let x = 0 end
(* 使用函子构造一个新的模块 B *)
module B = IncX(A)
let xinB = B.x
```

```
val xinB : int = 1
```

结果和我们的期望相符。

函子的定义有两种等价语法：

```
module F (M : S) = struct
  ...
end

module F = functor (M : S) -> struct
  ...
end
```

💡 提示

函数和函子非常相似。你之前见过的普通函数的定义也有两种等价的语法：

`let add x y = x + y` 和 `let add = fun x y -> x + y` 也是等价的。

和普通函数一样，函子也具备“高阶”的特质，这意味着你可以把多个模块作为参数（或者说每次接受一个模块作为参数，返回一个匿名的函子作为结果，这个匿名函子会继续应用到下一个参数模块）：

```
module F (M1 : S1) ... (Mn : Sn) = struct
  ...
end

module F = functor (M1 : S1) -> ... -> functor (Mn : Sn) -> struct
  ...
end
```

比如：

```
module type X = sig
  val x : int
end

module type Y = sig
  val y : int
end

module F (X : X) (Y : Y) = struct
  let f = X.x + Y.y
end
```

```
> module F : functor (X : X) (Y : Y) -> sig val f : int end
```

3.5.2 Functor Type Definition

你可以显式定义函子的类型：

```
module type FunctorType = functor (ParamModule : ParamSignature) ->
  ResultSignature
```

`ParamModule` 是参数模块，`ParamSignature` 和 `ResultSignature` 都是签名表达式。比如：

```
module type X = sig
  val x : int
end

module type Y = sig
  val y : int
end

(**
  显式定义函子的类型
  写成 module type FXY = functor (A : X) -> sig val y : int end 也可以
*)
module type FXY = functor (A : X) -> Y

module A : X = struct
  let x = 1
end

module F : FXY = functor (A : X) -> struct
  let y = A.x + 1
end

module B : Y = F(A)

let yinB = B.y;;

> val yinB : int = 2
```

3.5.3 Map

OCaml 标准库（`stdlib`）中的 `Map` 模块帮助你存储一组键值对。开发者使用二叉搜索树实现 `Map` 以提升检索的效率。一个显著的问题是**多态**：开发者无法预设用户正在使用的“键”是什么数据类型——它可能是简单的数字，比如（年龄，姓名），也可能是复杂的字符串；而且开发者也无法预设键之间的大小比较规则。

因此，OCaml 的 `Map` 模块提供了一个函子 `Make`。 `Make` 接受一个特定形式的模块作为输入，返回在给定模块基础上建立的 `Map`（二叉搜索树），作为参数输入的模块必须满足如下模块签名：

```
module type OrderedType = sig
  (* 键的类型 t *)
  type t
  (* 用于键之间比较大小关系的函数：当大小相同时返回 0，第一个参数小于第二个参数时返回负数，反之正数 *)
  val compare : t -> t -> int
end
```

`Make` 的返回模块支持对于一组键值对的大部分操作：

```
module type S = sig
  type key
  type 'a t
  val empty: 'a t
  val mem: key -> 'a t -> bool
  val add: key -> 'a -> 'a t -> 'a t
  val find: key -> 'a t -> 'a
  ...
end
```

💡 提示

`compare` 类似于你在 C++ 中使用 `sort` 进行排序时传入的比较函数 `cmp`，以及你在 Python 中使用 `sorted` 进行可迭代对象的排序时传入的比较函数 `cmp`。如果你对后两者也不太了解，可以参考 <https://www.runoob.com/python/python-func-sorted.html>

你可以尝试使用 `Map.Make` 存储一组形如“学号-姓名”的键值对：

```
(* 把学号定义为键值 *)
module StuNum = struct
  type t = int
  let compare x y = x - y
end
(* 建立 Map 索引 *)
module StuMap = Map.Make(StuNum)

(* 把一组键值对 (20233000000000, "Ella") 放进Map中 *)
let stumap = StuMap.add 2023300000000 "Ella" StuMap.empty
let name = StuMap.find 2023300000000 stumap;;

> val name : string = "Ella"
```

3.5.4 Set

OCaml 中的 `Set` 模块用于创建和操作不可变的有序集合，使用平衡二叉树实现。和 `Map` 模块类似，OCaml 在 `Set` 模块中也实现了一个函子 `Make`。参数模块需要满足下面的签名：

```
module type OrderedType = sig
  (* 键的类型 t *)
  type t
  (* 用于键之间比较大小关系的函数：当大小相同时返回 0，第一个参数小于第二个参数时返回负数，反之正数 *)
  val compare : t -> t -> int
end
```

3.5.5 “Functor”

你可能已经听说过 `Monad`，它往往在解决 I/O 问题的时候被使用。`Monad` 也是一个函子（`Functor`），但是和我们前文提到的函子是完全不同的概念，只是恰巧名字相同而已。在 OCaml 里，`Functor` 指的是作用于模块的函数，它接受一个模块作为参数，并返回一个新模块。在范畴

论和函数式编程（如 Haskell）中，Functor 是一种类型变换，即它是一个类型构造器，并且遵循某些映射规则：它将一种类型 A 映射到另一种类型 $F A$ 。它提供一个映射函数 `map`，能够将 $A \rightarrow B$ 映射为 $F A \rightarrow F B$ ，同时保持某些性质：`map id = id` `map (f ∘ g) = map f ∘ map g`。此外，函子 Functor 在其他高级语言中也有出现，含义也截然不同。

Monad 不在本次实验的内容之内，如果你感兴趣，参考：<https://cs3110.github.io/textbook/chapters/ds/monads.html>

4 Class and Object

① 注意

OCaml 中的 Module 和 Class 在某些地方上很相似：都提供了抽象和封装机制，以便实现子类型化和继承。二者也有很大的区别：Module 不能像 Class 那样实例化、Class 不能像 Module 那样在内部定义新的变体类型、Module 像是“静态”的而 Class 更像是“动态”的。你可以检索相关的信息（论坛、论文或者文档）了解更多。

4.1 Introduction

面向对象编程（Object Oriented Programming，OOP，面向对象程序设计）的主要思想是把构成问题的各个事务分解成各个对象，建立对象的目的不是为了完成一个步骤，而是为了描述一个事物在整个解决问题的步骤中的行为。面向对象编程的主要特征是封装、继承和多态。OCaml 支持面向对象的编程。你可以定义一个类来封装系统的内部实现，创建一个子类继承父类的关系及其重写父类上的方法（多态）。

4.2 Class Definition

4.2.1 Basic Class and Object

定义一个类的基本语法：

```
class name =  
  object  
    val ...  
    method ...  
  end
```

形象点说，`object ... end` 内定义了一个对象的内容，而关键字 `class` 给这一个对象做了一层“包装”，使一个对象变成一类对象，并且给这一类对象赋予一个名称，还是其具备继承等多种功能。`name` 是类名，`val` 后是类中可能被使用的绑定，`method` 后跟着类中定义的方法。你可以尝试定义一个非常简单的计数器类：

```
class counter =  
  object  
    val zero = 0  
    val mutable x = 0  
    method get = x  
    method add = x <- x + 1
```

```
method setzero = x <- zero
end;;
```

① 注意

因为 `x` 的值是可变的，我们需要添加关键字 `mutable` 来标识它

```
> class counter :
  object
    val mutable x : int
    val zero : int
    method add : unit
    method get : int
    method setzero : unit
  end
```

和很多高级语言一样，在定义类的时候你可以传入参数。此时 `class` 就和函数相似，接受一个参数，返回一个类：

```
class name = fun param ->
  object
    val ...
    method ...
  end

(* 等价写法 上面那种写法的语法糖 *)
class name param =
  object
    val ...
    method ...
  end
```

你可以尝试增强上面的计数器，现在给计数器添加一个初值：

```
class counter initial =
  object
    val zero = 0
    val mutable x = initial
    method get = x
    method add = x <- x + 1
    method setzero = x <- zero
  end;;
```

使用关键字 `new` 实例化一个对象，并且使用 `#` 调用类上的方法：

```
(* 创建一个新对象 *)
let count = new counter 3;;
(* 调用 add 方法 *)
count#add;;
let result = count#get;;
```

```
> val count : counter = <obj>
> - : unit = ()
> val result : int = 4
```

对象内的方法可以调用该对象内的其他方法，这需要显式的写出 `object(self)`：

```

class pokemon =
  object(self)
    (* 生命 *)
    val mutable hp = 100
    (* 魔力量 *)
    val mutable mp = 10
    (* 进行攻击 需要知道还能放什么技能 *)
    method attack = print_endline ("attack! " ^ self#skill)
    (* 计算能不能放技能 *)
    method skill = match mp with
      | 0 -> "no mp"
      | _ -> mp <- mp - 1; "fireball"
    end;;

let charmander = new pokemon;;

charmander#attack

> attack! fireball

```



4.2.2 Immediate Object

在某些情况下，你可以不通过类创建一个对象，直接使用 `object` 关键字：

```

let p =
  object
    val mutable x = 0
    method get_x = x
    method move d = x <- x + d
  end;;

val p : < get_x : int; move : int -> unit > = <obj>

```

`p` 已经是一个对象，不需要再次实例化。你可以直接调用 `p` 上的方法：

```

p#get_x;;

> -: int = 0

```

用这种方式定义的对象叫做直接对象（Immediate Object）。与类相比，直接对象不能被继承。

⚠ 小心

与类不能在表达式中定义这一点不同，直接对象可以出现在任何地方，并使用它们所在环境中的变量。

```

let minmax x y =
  if x < y then object method min = x method max = y end
  else object method min = y method max = x end;;

```

`minmax` 接受两个参数，然后返回一个对象

4.2.3 Polymorphic Class

你已经尝试用模块实现过栈。下面是用类实现的栈的一个版本：

```
class stack_of_ints =
  object (self)
    val mutable the_list = ([] : int list)      (* instance variable *)
    method push x =                             (* push method *)
      the_list <- x :: the_list
    method pop =                                (* pop method *)
      let result = List.hd the_list in
      the_list <- List.tl the_list;
      result
    method peek =                               (* peek method *)
      List.hd the_list
    method size =                               (* size method *)
      List.length the_list
  end;;
```

一个显著的区别是，因为使用了参数化变体 `type 'a t = 'a list`，所以用模块实现的栈支持存放多种数据类型的数据。类似 `'a list`，你可以使用 `'a stack` 来使栈支持多种数据类型。比如：

```
class ['a] stack =
  object (self)
    val mutable list = ([] : 'a list)          (* instance variable *)
    method push x =                            (* push method *)
      list <- x :: list
    method pop =                               (* pop method *)
      let result = List.hd list in
      list <- List.tl list;
      result
    method peek =                             (* peek method *)
      List.hd list
    method size =                             (* size method *)
      List.length list
  end;;
```

类的名称前的 `[ ... ]` 绑定了类对应的类型参数。`['a]` 这个类型参数表明 `stack` 类是一个泛型类，也叫多态类，是一组对应具体数据类型的类的类（“class of classes”），它可以存储任意类型的元素。

如果你不显式地在类名前加上 `['a]`，尝试仅仅把 `list` 定义成 `'a list` 来实现多态性，你会遇到报错：

```
> Error: The type variable 'a is unbound in this class.
```

这是由 OCaml 的类型系统决定的。在 OCaml 的类中，实例变量不能直接是未绑定的多态类型，你需要把 `'a` 绑定到类的类型参数上。

如果你在某个类中实现了一个多态方法，即使你没有声明多态的类上变量（比如上文的 `'a list`），你仍然需要在类上绑定类型参数。如果你这么写会有报错：

```
class listExec =
object
  (* 显然这是一个多态方法 *)
  method add x l = x::l
end;;
```

```
> Error: Some type variables are unbound in this type
```

你必须绑定类型变量：

```
class ['a] listExec =
object
  method add (x:'a) (l:'a list) = x::l
end;;
```

你可以尝试使用多态类 `stack`：

```
let s = new stack;;
s#push 1.0;;
s#push 3.0;;
s#pop;;
```

```
> - : float = 3.
```

通过向栈中压进两个浮点数，类型系统推断出这是一个浮点类型的栈。你也可以在实例化栈的时候显式的定义其类型：

```
let s : int stack = new stack;;
```

💡 提示

OCaml 和多态类/泛型类和 C++ 的泛型机制有点相似。

OCaml 支持你定义多个类型参数，比如：

```
class ['a, 'b] pair =
object
  val mutable fst = (None : 'a option)
  val mutable snd = (None : 'b option)
  method set x y = fst <- Some x; snd <- Some y
  method get = (fst, snd)
end;;
```

这里 `['a, 'b]` 表示 `pair` 类有两个类型参数，允许存储两种不同类型的值。

4.3 Inheritance

4.3.1 Virtual Method

你可以使用关键字 `virtual` 来声明一个虚方法(虚变量)。和很多高级语言的虚成员一样，你不必在声明时给出具体实现，具体实现在随后的子类中给出。包含虚方法的类也必须用 `virtual` 来标记，并且不能被实例化，也就是说，不能创建该类的任何对象。比如：

```
class virtual pokemon =
object
  (* 宝可梦的名字 *)
```

```

val virtual name : string
(* 宝可梦的生命值 *)
val virtual hp : int
(* 宝可梦的攻击力 *)
val virtual atk : int

(* 获取宝可梦的名字 *)
method get_name = name
(* 获取宝可梦的生命值 *)
method get_hp = hp
(* 根据给出的对方宝可梦信息计算实际伤害 *)
method virtual attack : pokemon -> int
end;;

```

被标记为 `virtual` 的类也可以支持多态，你依然可以使用 `['a, 'b, ...]` 来绑定和使用类型变量。

4.3.2 Inherit

你可以使用 `inherit` 关键字来继承另一个类（称之为父类），并且给出父类上的虚方法的实现。

```

class virtual animal =
  object
    (* 虚方法 *)
    method virtual speak : unit
  end

class dog =
  object
    (* 继承自 animal 类 *)
    inherit animal
    (* 实现父类上的虚方法 *)
    method speak = print_endline "Woof!"
  end

```

如果你想要使用父类上已经实现的属性和方法，当你使用简单的 `inherit` 继承时，父类的方法会被直接合并到子类中，通过显式的定义 `self` 后直接通过 `self#` 调用。如果你想要区分子类和父类，可以在继承时使用关键字 `as` 给父类起一个别名，然后通过该别名来调用。例如：

```

class base =
  object
    (* 定义一个私有变量 *)
    val mutable counter = 0
    method get_counter = counter
    method incr = counter <- counter + 1
    method print_counter = print_endline (string_of_int counter)
  end

class derived =
  object(self)
    inherit base as super (* 给父类起一个别名 super *)
    method reset_and_print =
      (* 调用父类的方法 *)
      super#incr;
      super#print_counter;
  end

```

```
print_endline ("Current counter: " ^ string_of_int (super#get_counter))
end
```

如果你想要重写父类上已经实现的方法，不需要任何关键字，只需要在子类中直接重新定义：

```
class base =
  object
    method greet = print_endline "Hello from base"
  end

class derived =
  object(self)
    inherit base
    method greet = print_endline "Hello from derived"
  end
```

⚠ 小心

当你重写父类上的方法后，在不给予父类一个别名的情况下再次使用 `self#` 试图调用父类上的方法时，你只会调用到子类上重写后的方法。只有给予父类一个别名并且通过别名才可以正确调用父类上的方法。

在 OCaml 中，多继承允许一个类同时继承多个父类的行为和属性（称为“多继承”）。实现方式是通过在类定义中连续使用多个 `inherit` 语句。当多个父类重复定义了一个同名方法后，在不使用别名的情况下会保留最后一个继承的类中的实现。例如：

```
class parent1 =
  object
    method greet = print_endline "Hello from parent1"
  end

class parent2 =
  object
    method greet = print_endline "Hello from parent2"
    method farewell = print_endline "Goodbye from parent2"
  end

class child =
  object(self)
    inherit parent1
    inherit parent2
    method greet_all =
      print_endline "Calling greet:";
      self#greet;
      print_endline "Calling farewell:";
      self#farewell
  end
```

```
Calling greet:
Hello from parent2
Calling farewell:
Goodbye from parent2
```

4.3.3 Class Interface

类的接口（Interface）类似于模块签名，使用 `class type` 来显式的定义一个类的接口：

```
(* 定义一个接口，包含两个方法 *)
class type my_interface =
object
  method method1 : int -> int
  method method2 : string -> unit
end
```

可以在类定义时通过 `:` 指定接口：

```
(* 实现 my_interface 接口 *)
class my_class : my_interface =
object
  method method1 x = x + 1
  method method2 s = print_endline s
end
```

接口中声明的方法就是你希望向外暴露的内容。当你实现一个类时，可以实现更多的方法，但如果你在外部只使用接口类型来引用这个类，那么只有接口中声明的方法可见。比如：

```
class type public_interface =
object
  method foo : int -> int
  method bar : string -> unit
end
class my_class =
object
  method foo x = x + 1
  method bar s = print_endline s
  method baz = print_endline "This is internal"
end

(* 把 my_class 的实例限定为 public_interface 类型 *)
let instance : public_interface = new my_class
```

在上面的例子中，虽然 `my_class` 中实现了 `baz` 方法，但由于 `baz` 不在 `public_interface` 接口中，外部代码通过 `instance` 只能调用 `foo` 和 `bar` 方法。

4.3.4 Privacy

使用关键字 `private` 定义私有方法。私有方法不会暴露给外部（不会在类的接口中出现），它们只能被同一对象的其他方法调用：

```
class my_class =
object (self)
  method public_method =
    print_endline "This is a public method.";
    (* 在类内部调用私有方法 *)
    self#private_method

  method private private_method =
    print_endline "This is a private method."
end

let () =
```

```
let obj = new my_class in
obj#public_method;
(* 以下调用将导致编译错误，因为 private_method 是私有的 *)
(* obj#private_method *)
```

私有方法是会被子类继承的(默认情况下它们在子类中是可见的), 除非它们被接口隐藏。同时, 私有方法可以在子类中被转化为公有: 只需要重新定义该方法。

4.3.5 Oo Module

OCaml 的 **Oo 模块** 是整个对象系统的核心组成部分:

- 克隆对象 `Oo.copy`

该函数能生成一个对象的浅拷贝, 也就是说, 它返回一个新的对象, 其方法和实例变量初始值与原对象相同。将新值赋给副本的实例(使用方法调用)不会影响原始实例。在类中, 你可以利用它来实现类似“复制”或“更新”对象的功能。例如, 如果你想在类方法中返回一个修改后的新对象, 而不直接改变原对象, 可以这样写:

```
class my_class =
  object (self)
    (* 其它方法和属性 *)
    method copy = Oo.copy self
  end
```

- 标识对象 `Oo.id`

返回一个整数, 这个整数唯一标识当前运行时中一个对象。这对于判断两个对象是否为同一物理实例(即使它们在结构上看起来一样)非常有用。`=` 和 `<>` 可用于比较对象的物理相等性(即一个对象与其拷贝在物理上并不相同)。你还可以使用 `<` 等操作符, 它们提供了一个基于对象 ID 的排序。

5 Extend

5.1 Compile

5.1.1 Dune

对于一个比较大的项目, 继续使用 Utop 会降低你的开发效率。你可以尝试使用 Dune 来辅助构建一个 OCaml 项目。

参考 Dune 的官方文档 [Website](#)

参考 OCaml 的官方文档 [Website](#)

5.2 Test

5.2.1 OUnit2

函数式编程相比命令式编程, 调试难度更大。构建单元测试来验证你的函数是否正确是非常必要的, 你可以尝试使用 OUnit2 来构建单元测试。

参考 Cornell cs3110 Textbook [Website](#)

参考官方文档 [Website](#)